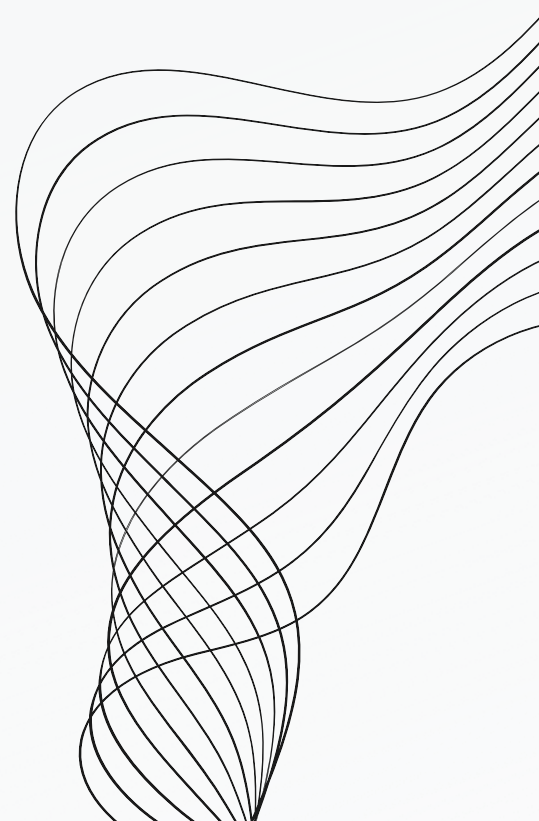
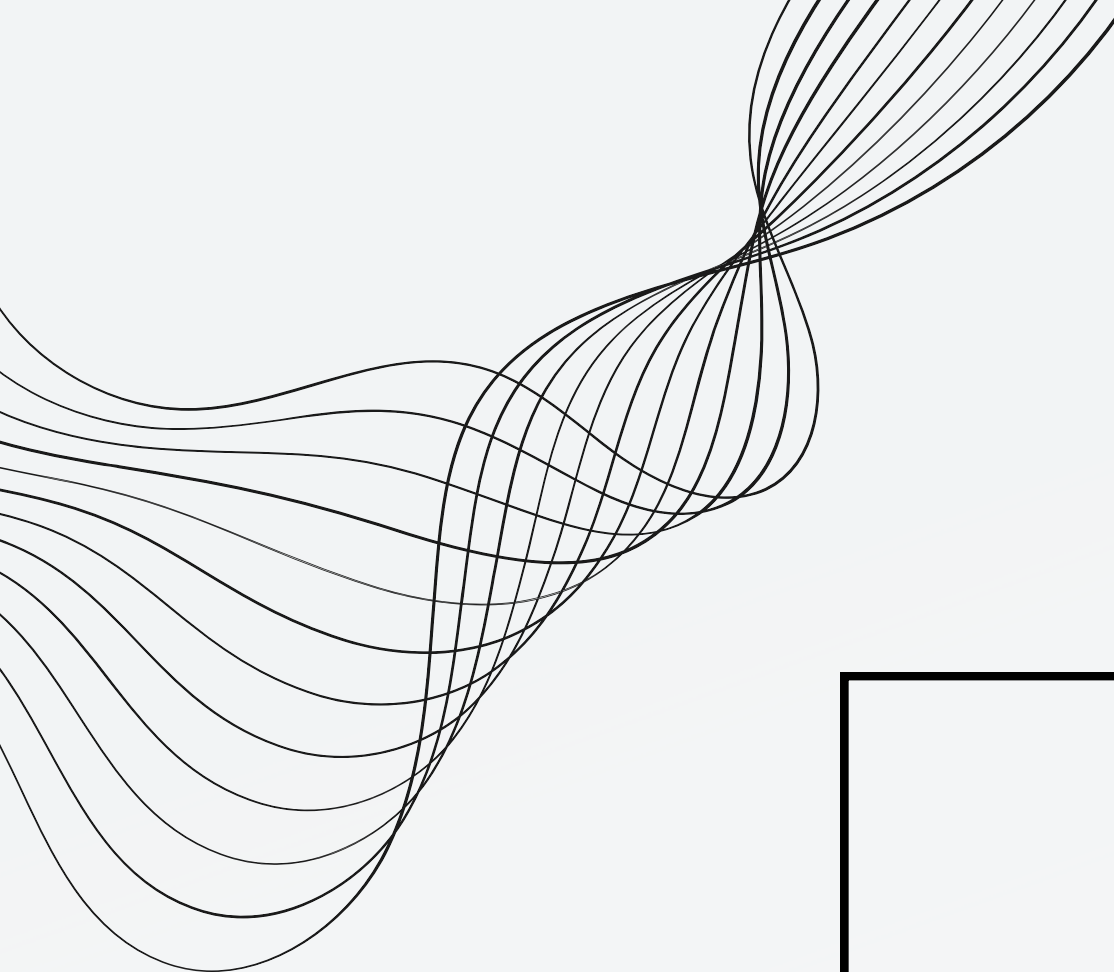
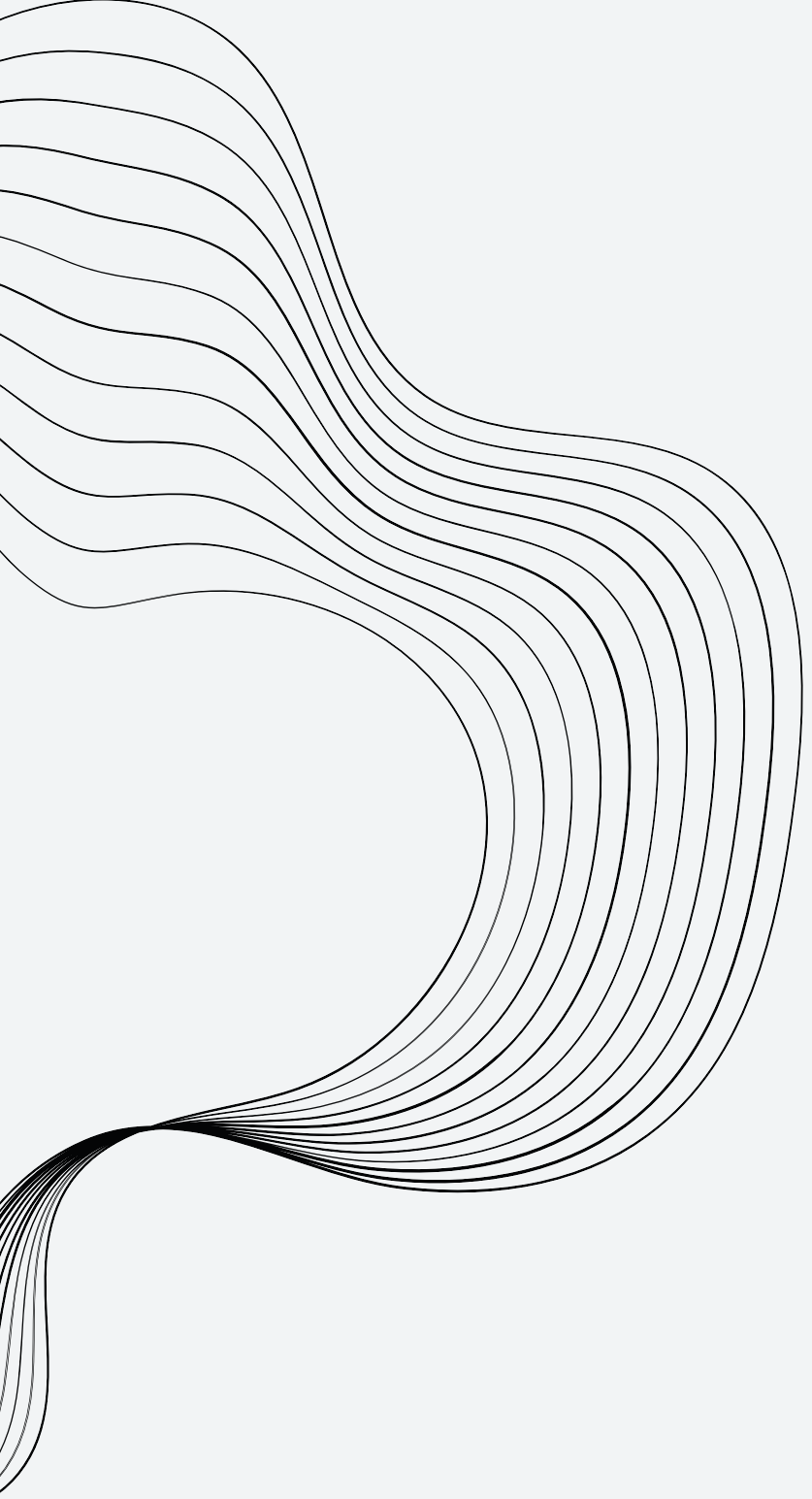




U.T.4

PROGRAMACIÓN CON FUNCIONES, ARRAYS Y OBJETOS DEFINIDOS POR EL USUARIO

DESARROLLO WEB EN ENTORNO CLIENTE



CONTENIDO

01

FUNCIONES

02

ARRAYS

03

OBJETOS





01

FUNCIONES

¿QUÉ ES UNA FUNCIÓN?

Las funciones nos permiten agrupar líneas de código en tareas con un nombre, para que, posteriormente, podamos hacer referencia a ese nombre para realizar todo lo que se agrupe en dicha tarea.

Para usar funciones hay que hacer 2 cosas:

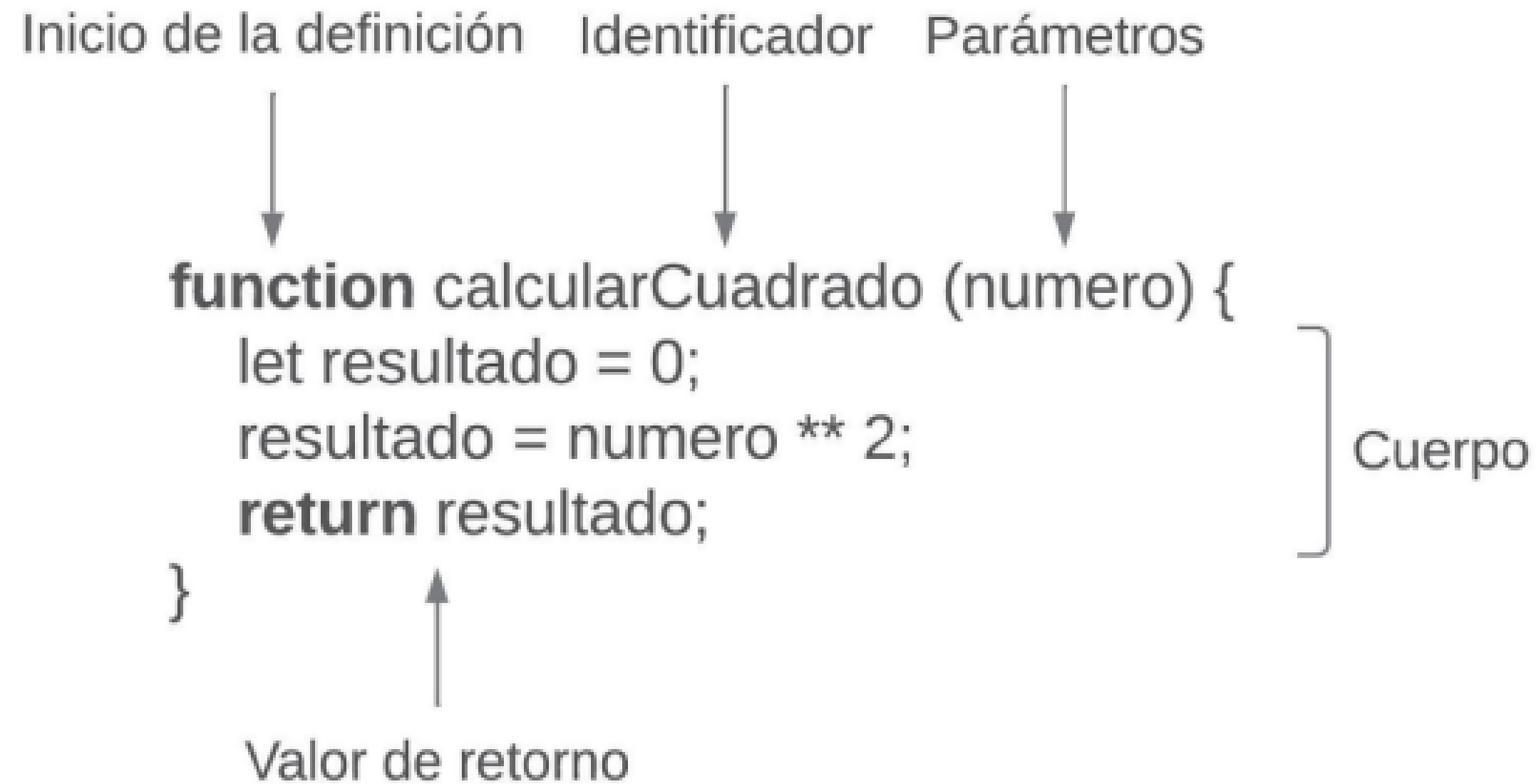
- Declarar la función: Preparar la función, darle un nombre y decirle las tareas que realizará.
- Ejecutar la función: «Llamar» a la función para que realice las tareas de su contenido.

```
// Declaración de la función "saludar"  
function saludar() {  
  // Contenido de la función  
  console.log("Hola, soy una función");  
}
```

```
// Ejecución de la función  
saludar();
```

ANATOMÍA DE UNA FUNCIÓN

Diagrama con los elementos constitutivos de una función en notación declarativa:



PARÁMETROS DE UNA FUNCIÓN

Los parámetros o argumentos de las funciones son valores que se pasan a una función para ser procesados. Son los valores que permiten la comunicación con las funciones.

PARÁMETROS POR VALOR Y POR REFERENCIA

En JavaScript, los parámetros pueden pasarse a las funciones por valor o por referencia, dependiendo del tipo de datos que se esté manipulando.

PASO POR VALOR (PRIMITIVOS)

Cuando pasas un valor primitivo (como números, cadenas, booleanos) como argumento a una función, se pasa por valor. Esto significa que se crea una copia del valor original dentro de la función, y cualquier modificación en la copia no afectará al valor original.

```
function duplicar(x) {  
  x = x * 2;  
  console.log("Dentro de la función:", x);  
}  
  
let numero = 5;  
duplicar(numero);  
console.log("Fuera de la función:", numero);
```

PASO POR REFERENCIA (OBJETOS Y ARRAYS)

Cuando pasas un objeto (incluyendo arrays) como argumento a una función, se pasa por referencia. Esto significa que la función recibe una referencia al objeto original, no una copia. Cualquier modificación en el objeto dentro de la función afectará al objeto original.

```
function agregarElemento(arr, elemento) {  
  arr.push(elemento);  
  console.log("Dentro de la función:", arr);  
}  
  
let miArray = [1, 2, 3];  
agregarElemento(miArray, 4);  
console.log("Fuera de la función:", miArray);
```


PARÁMETROS DE UNA FUNCIÓN

PARÁMETROS VARIABLES

Los parámetros variables (o argumentos variables) en JavaScript se refieren a la capacidad de una función para recibir un número variable de argumentos.

OPERADOR REST(...)

Esta funcionalidad se logra mediante el uso del operador rest (...). Los parámetros variables permiten que una función acepte cualquier cantidad de argumentos y los trate como un array en su implementación.

```
function suma(...numeros) {  
  let resultado = 0;  
  for (const numero of numeros) {  
    resultado += numero;  
  }  
  return resultado;  
}  
  
console.log(suma(1, 2, 3, 4, 5)); // Devuelve 15
```

OBJETO ARGUMENTS

Otra opción sería utilizar un objeto disponible en Javascript llamado arguments, que contiene un array que almacena todos los argumentos que se han indicado en la invocación a la función.

```
function suma() {  
  let resultado = 0;  
  for (let i = 0; i < arguments.length; i++) {  
    resultado += arguments[i];  
  }  
  return resultado;  
}  
  
console.log(suma(1, 2, 3, 4, 5)); // Devuelve 15  
console.log(suma(10, 20));        // Devuelve 30
```

TIPOS DE FUNCIONES

En JavaScript, hay varios tipos de funciones según su uso y definición:

FUNCIONES POR DECLARACIÓN

Probablemente, la forma más popular, Permite declarar una función que existirá a lo largo de todo el código

- Se pueden llamar antes de su definición en el código

```
function suma(a, b) {  
    return a + b;  
}
```

FUNCIONES POR EXPRESIÓN

Estas funciones se definen como expresiones y se pueden asignar a variables.

- No se pueden llamar antes de su definición en el código.

```
const resta = function(a, b) {  
    return a - b;  
};
```


TIPOS DE FUNCIONES

FUNCIONES COMO OBJETOS

Se pueden declarar funciones como si fueran objetos. Sin embargo, es un enfoque que no se suele utilizar en producción

```
const saludar = new Function("return 'Hola';");  
  
saludar(); // 'Hola'
```

FUNCIONES ANÓNIMAS

Las funciones anónimas no tienen un nombre asociado y se pueden definir en el lugar donde se necesitan, por ejemplo, como argumentos de otras funciones.

```
setTimeout(function() {  
    console.log("¡Hola, mundo!");  
}, 1000);
```

```
// Función anónima "saludo"  
const saludo = function () {  
    return "Hola";  
};
```

TIPOS DE FUNCIONES

FUNCIONES CALLBACK

Un callback es una función que se pasa como argumento a otra función y se ejecuta después de que se complete una tarea específica. Los callbacks son una parte fundamental de la programación asíncrona en JavaScript y se utilizan para manejar eventos, solicitudes de red, temporizadores y otras operaciones asíncronas.

```
function operacionAsincronica(x, y, callback) {  
    setTimeout(function() {  
        const resultado = x + y;  
        callback(resultado);  
    }, 1000);  
}  
  
function mostrarResultado(resultado) {  
    console.log("El resultado es: " + resultado);  
}  
  
operacionAsincronica(5, 3, mostrarResultado);  
console.log("Esperando el resultado...");
```

TIPOS DE FUNCIONES

FUNCIONES FLECHA (ARROW FUNCTIONS)

Son una forma corta de escribir funciones que aparece en Javascript a partir de ECMAScript 6. Básicamente, se trata de reemplazar la palabra function y añadir => antes de abrir las llaves

```
const func = function () {  
  return "Función tradicional.";  
};
```

```
const func = () => {  
  return "Función flecha.";  
};
```

SIMPLIFICAR EL CÓDIGO

- Si el cuerpo de la función sólo tiene una línea, podemos omitir las llaves ({}).
- Además, en ese caso, automáticamente se hace un return de esa única línea, por lo que podemos omitir también el return.
- En el caso de que la función no tenga parámetros, se indica como en el ejemplo anterior: () =>.
- En el caso de que la función tenga un solo parámetro, se puede indicar simplemente el nombre del mismo: e =>.
- En el caso de que la función tenga 2 ó más parámetros, se indican entre paréntesis: (a, b) =>.

```
const func = () => "Función flecha."; // 0 parámetros: Devuelve "Función flecha"  
const func = (e) => e + 1; // 1 parámetro: Devuelve el valor de e + 1  
const func = e => e + 1; // 1 parámetro: Devuelve el valor de e + 1  
const func = (a, b) => a + b; // 2 parámetros: Devuelve el valor de a + b
```

TIPOS DE FUNCIONES

FUNCIONES AUTOEJECUTABLES

Son funciones que se ejecutan inmediatamente después de su definición. Se utilizan para crear un ámbito privado de variables.

```
// Función autoejecutable  
(function () {  
  console.log("Hola!!");  
})();
```

RECURSIVIDAD

La recursividad en JavaScript es un concepto en el que una función se llama a sí misma para resolver un problema. Esto es útil cuando se enfrenta a problemas que se pueden dividir en casos más pequeños y similares. Para usar la recursividad en JavaScript, debes tener dos componentes clave:

1. Caso Base: Este es el punto de parada de la recursión. Cuando se alcanza el caso base, la función recursiva deja de llamarse a sí misma y comienza a devolver valores.
2. Caso Recursivo: En esta parte, la función se llama a sí misma con un conjunto de argumentos modificados. Estos argumentos modificados deben acercarte al caso base en cada llamada.

```
function factorial(n) {  
  // Caso base: cuando n es 0, el factorial es 1.  
  if (n === 0) {  
    return 1;  
  }  
  // Caso recursivo: llamamos a la función con n-1 y multiplicamos por n.  
  else {  
    return n * factorial(n - 1);  
  }  
}  
  
console.log(factorial(5)); // Devuelve 120 (5 * 4 * 3 * 2 * 1)
```



La recursividad es útil para resolver problemas complejos de manera elegante, pero también es importante comprenderla completamente y diseñarla cuidadosamente para evitar problemas de rendimiento.